

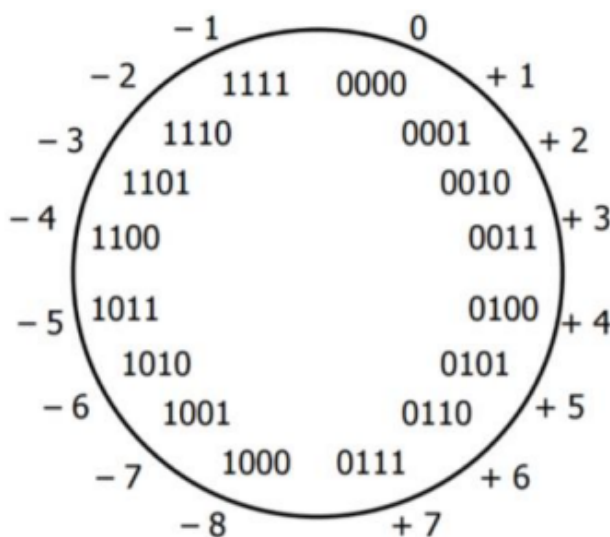
Quiz1 of Computer Architecture (2021 Fall)

Solutions

Problem A

~~Two's Complement~~ (~~https://en.wikipedia.org/wiki/Two%27s_complement~~) is the standard for representing signed integers:

- The most significant bit (MSB) has a negative value; all others have positive values (same as unsigned)
- Binary addition is performed the same way for signed and unsigned
- The bit representation for the negative (additive inverse) of a two's complement number can be found by:
 - flipping all the bits and adding 1 (i.e. $-x = \sim x + 1$).



The above "number wheel" showing the relationship between 4-bit numerals and their Two's Complement (https://en.wikipedia.org/wiki/Two%27s_complement) interpretations is shown on the right:

- The largest number is 7 whereas the smallest number is -8
- There is a nice symmetry between numbers and their negative counterparts except for -8

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer **Yes** / **No** to the following questions:

1. Is it possible to get an overflow error in Two's Complement (https://en.wikipedia.org/wiki/Two%27s_complement) when adding numbers of opposite signs?

- A01 = ?

No .

Overflow errors only occur when the correct result of the addition falls outside the range of $[-(2^{n-1}), 2^{n-1} - 1]$. Adding numbers of opposite signs will not result in numbers outside of this range.

2. If you interpret a N bit Two's Complement (https://en.wikipedia.org/wiki/Two%27s_complement) number as an unsigned number, would negative numbers be smaller than positive numbers?

- A02 = ?

No

In Two's Complement, the MSB is always 1 for a negative number. This means ALL Two's Complement negative numbers will be larger than the positive numbers.

3. If you interpret an N bit Bias notation (https://ocw.mit.edu/courses/aeronautics-and-astronautics/16-01-unified-engineering-i-ii-iii-iv-fall-2005-spring-2006/comps-programming/number_systems.pdf) number as an unsigned number (assume there are negative numbers for the given bias), would negative numbers be smaller?

- A03 = ?

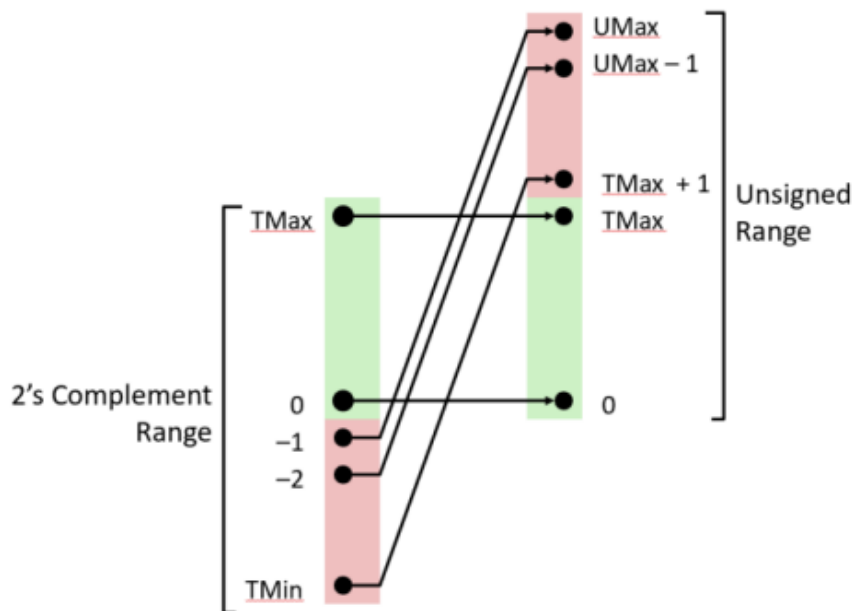
Yes

In bias notation, we add a bias to the unsigned interpretation to create the value. This means that negative numbers will stay smaller than positive numbers. This is unlike Two's Complement.

Problem B

Arithmetic overflow occurs when the result of a calculation can't be represented in the current encoding scheme (i.e., it lies outside of the representable range of values), resulting in an incorrect value.

- Unsigned overflow: the result lies outside of $[UMin, UMax]$; an indicator of this is when you add two numbers and the result is smaller than either number.
- Signed overflow: the result lies outside of $[TMin, TMax]$; an indicator of this is when you add two numbers with the same sign and the result has the opposite sign.



1. Find the largest 8-bit unsigned numeral c (answer in HEX) such that $c + 0x80$ causes NEITHER signed nor unsigned overflow in 8 bits.

• B01 = ?

0x7F

Unsigned overflow will occur for $c > 0x80$. Signed overflow can only happen if c is negative (also $> 0x80$). Largest is therefore, 0x7F

2. Find the smallest 8-bit numeral c (answer in HEX) such that $c + 0x71$ causes signed overflow, but NOT unsigned overflow in 8 bits.

• B02 = ?

0xF

For signed overflow, need $(+) + (+) = (-)$. For no unsigned overflow, need no carryout from MSB. The first $(-)$ encoding we can reach from $0x71$ is $0x80$. $0x80 - 0x71 = 0xF$.

Problem C

According to IEEE 754 Floating Point Standard, the value of a real number can be represented in scientific binary notation as:

$$Value = (-1)^{sign} \times Mantissa_2 \times 2^{Exponent} = (-1)^S \times 1.M_2 \times 2^{E-bias}$$

The binary representation for floating point values uses three fields:

- s : encodes the sign of the number (0 for positive, 1 for negative)
- E : encodes the exponent in **biased notation** with a bias of $2^{w-1} - 1$
- M : encodes the mantissa (or significand, or fraction) – stores the fractional portion, but does not include the implicit leading 1 .

	S	E	M
float	1 bit	8 bits	23 bits
double	1 bit	11 bits	52 bits

1. Let's say that we want to represent the number 3145728.125_{10} (broken down as $2^{21} + 2^{20} + 2^{-3}$). Is it enough to represent this number single precision floating point? (Please answer **Yes / No**)

- C01 = ?

No

Could only represent $2^{21} + 2^{20}$. Not enough bits in the mantissa to hold 2^{-3} , which caused rounding.

2. What is the decimal value of float $0x80000000$? (Answer with leading + and -)

- C02 = ?

-0

3. What is the decimal value of float $0xFF94BEEF$? (Answer with leading + and -)

- C03 = ?

NaN

4. What is the decimal value of float $0x41180000$? (Answer with leading + and -)

- C04 = ?

+9.5

$0x41180000 = 0b\ 0|100\ 0001\ 0|001\ 1000\ 0\dots0$.

$S = 0, E = 128 + 2 = 130 \rightarrow \text{Exponent} = E - \text{bias} = 3, \text{Mantissa} = 1.0011_2$

$$1.0011_2 \times 2^3 = 1001.1_2 = 8 + 1 + 0.5 = 9.5$$

5. What is the smallest positive value that can be stored using a single precision float?
Answer in HEX value. (video: [How to Calculate Smallest Float Value in IEEE 754 Standard \(Single Precision\)](https://youtu.be/SMLcrWgE2sk).(<https://youtu.be/SMLcrWgE2sk>.)

- C05 = ?

0x00000001

$$0x00000001 = 2^{-23} \times 2^{-126}$$

6. What is the smallest positive normalized value that can be stored using a single precision float?

- C06 = ?

0x00800000

$$0x00800000 = 2^{-126}$$

7. What is the decimal value of float 0xFF800000 ? (Answer with leading + and -)

- C07 = ?

$-\infty$

8. What is the decimal value of float 0x421E4000 ? (Answer with leading + and -)

- C08 = ?

+39.5625

Problem D

Floating Point Mathematical Properties

- Not associative: $(2 + 2^{50}) - 2^{50} \neq 2 + (2^{50} - 2^{50})$
- Not distributive: $100 \times (0.1 + 0.2) \neq 100 \times 0.1 + 100 \times 0.2$
- Not cumulative: $2^{25} + 1 + 1 + 1 + 1 \neq 2^{25} + 4$

If x and y are variable type `float`, will the expression $(x+2*y)-y==x+y$ always be evaluated as true? (Please answer **Yes** / **No** with explanation.)

D01 = ?

No, Rounding error / Overflow (後者只要一個解釋符合就給分)

Problem E

Compute the decimal result of the following arithmetic expressions involving **6-bit Two's Complement** (https://en.wikipedia.org/wiki/Two%27s_complement), numbers as they would be calculated on a computer. Do any of these result in an overflow? Are all these operations possible?

1. Input: $0b100011 + 0b111010$

- E01 = ?

Overflow

Adding together we get $0b1011101$, however since we are working with 6-bit numbers we truncate the first digit to get $0b011101 = 29$. Since we added two negative numbers and ended up with a positive number, this results in an overflow.

2. Input: $0xFF - 0xAA$

- E02 = ?

Impossible

This is not possible, as these hex numbers would need 8 bits to represent and we are working with 6 bit numbers.

3. Input: $0x3B + 0x06$

- E03 = ?

1

Converting to binary, we get $0b111011 + 0b000110 =$ (after truncating) $0b000001 = 1$.

Despite the extra truncated bit, this is not an overflow as $-5 + 6$ indeed equals 1

Problem F

1. How do you write the bitwise exclusive-nor (XNOR) (https://en.wikipedia.org/wiki/XNOR_gate) operator in C program?

- F01 = ?

$x == y$ (只要有比較就給分)

2. Given `x` as an unsigned integer. Please use bitwise operators and `+`, `-`, `==` to check if `x` is a power of 2. Write down the expression in C without any branches (i.e., `if`, `else`, `?:`, `do`, `while`, `for`, `goto`)

• F02 = ?

`x & (x - 1) == 0`

3. The following function `absf` returns absolute value of a single precision float. What is the value of `F03`? Answer in HEX.

```
#include <stdint.h>
float absf(float x) {
    uint32_t mask = F03;
    union {
        uint32_t i;
        float f;
    } u = {.f = x};
    u.i &= mask;
    return u.f;
}
```

• F03 = ?

`0x7fffffff`